PRODUCT REQUIREMENTS DOCUMENT

NEURODESK AI

AI Notebook + Chatbot + Memory + Multi-Agent + Knowledge Graph

Phased product and technical specification | Free-first implementation | No Docker

Document field	Value
Version	1.2
Status	Draft for product and engineering alignment
Prepared	7 November 2026
Product scope	Web-based personal/workspace AI knowledge assistant
Primary constraint	No Docker or container-based development/deployment
Source architecture	NEURODESK AI — System Architecture (Free-First Approach) supplied with request

Stack interpretation The supplied architecture contains several alternatives. This PRD selects one default for each MVP capability and records the other pictured options as fallbacks or later choices. This prevents parallel implementations while retaining the free-first intent.

1. Executive summary

NeuroDesk AI is a browser-based workspace where users can upload and organize source material, ask questions grounded in that material, create and summarize notes, retain user-controlled memory, inspect concepts and relationships in a knowledge graph, and invoke focused agents and tools for research and action-oriented outputs. The default deployment runs on the user's local machine: web client, Node API, Python AI service, PostgreSQL, Redis, Neo4j, ChromaDB persistence, local file storage, and Ollama. Cloud inference and third-party integrations are optional and disabled by default.

The system is divided into a React/Next.js client, a Node.js/Express API and business-services layer, a Python AI orchestration service, retrieval and graph knowledge systems, persistence and cache services, model providers, real-time communications, and external integrations. It is delivered in gated phases so the core notebook and grounded chat exist before multi-agent and advanced integrations are added.

Non-negotiable deployment constraint Docker, Docker Compose, and container-based development or deployment are out of scope. The required MVP mode runs services locally as native processes; no cloud account, hosted database, cloud LLM, or paid API is required for core notebook, local RAG, graph, memory, and chat features.

2. Product vision, objectives, and non-goals

2.1 Vision

Provide one private, understandable workspace where a user can bring documents and web/video sources together, ask questions across them, build durable context over time, and see how the underlying concepts connect.

2.2 Objectives

- Deliver reliable, source-grounded answers over a user's uploaded documents and notes.
- Make ingestion, chat, notes, memory, and knowledge graph available from one workspace.
- Support a local-first default model path through Ollama and optional low-cost cloud inference fallbacks.
- Make multi-agent work inspectable: show intent, selected agent/tool, source evidence, and result.
- Protect workspace isolation and user control over files, memory, and generated knowledge.
- Ship incrementally with measurable exit criteria and no Docker dependency.

2.3 Non-goals for initial release

- A general-purpose autonomous agent that can take irreversible actions without user review.
- Enterprise SSO, complex role hierarchies, audit/compliance certifications, or multi-region HA.
- Training or fine-tuning foundation models; the system uses inference APIs and local models.
- Real-time collaborative editing or shared multi-user graph mutation.
- Guaranteed zero-cost hosting at scale; free tiers and quotas are provider-controlled and may change.
- Native mobile applications, browser extensions, or a public marketplace for agents.

3. Target users and primary jobs

Persona	Need / job to be done	Success signal
Individual researcher / student	Ingest PDFs, web pages, images, or video links; ask cross-source questions; preserve notes.	Can get a cited answer and revisit its source context.
Knowledge worker	Summarize material, extract key points, create plans and drafts from workspace context.	Turns source material into useful artifacts with fewer manual steps.
Power user / builder	Inspect memory, graph links, agents, and model routing; control local vs cloud inference.	Understands and can correct what the system remembers or invokes.

4. Product principles and key differentiators

- Memory depth: long-term user/workspace memory is represented in relational records, vector retrieval, and graph connections—not hidden solely in prompt history.
- Graph-based understanding: concepts and entities can be inspected as nodes and relationships in Neo4j.
- Offline-capable inference: Ollama is the primary local model interface; cloud use is opt-in/configurable.
- Actionable outputs: answers can lead to user-approved artifacts such as plans, roadmaps, code snippets, or study notes.
- Multi-modal, multi-source knowledge: documents, images, links, and YouTube transcripts share a common ingestion/retrieval path.
- Explainable orchestration: show the route/agent/tools used, the evidence returned, and the limits of the result.

5. Scope and functional requirements

ID	Capability	Requirement / expected behavior	Priority
FR-01	Authentication & workspace	User can create/access an account, maintain profile/preferences, and operate within an isolated personal workspace. JWT-backed sessions; password/session handling must follow secure practices.	P0
FR-02	Chat	Ask questions in a conversation, receive streaming responses, view citations/source snippets, and continue or start a new thread.	P0
FR-03	File ingestion	Upload PDF, DOCX/TXT/Markdown, common images, and source links; validate type/size; extract metadata/text; report processing status and errors.	P0
FR-04	Retrieval	Chunk and embed eligible content; retrieve relevant passages; build context with source IDs and citations before answer generation.	P0
FR-05	Notebook & notes	Create, edit, summarize, and delete user notes; associate notes with documents, concepts, and conversations.	P0
FR-06	Memory	Store user-approved preferences and useful conversation facts; let user inspect, correct, and delete memory. Avoid silently promoting sensitive data.	P1
FR-07	Knowledge graph	Create/query entities and relationships; link graph items back to source documents; render an interactive graph view.	P1
FR-08	Agent routing	Classify intent, select a bounded agent/tool, compose a task plan, execute, and synthesize results. Provide a safe fallback to standard chat.	P2
FR-09	Research tools	Support web search, calculator, file reader, YouTube transcript, and image analysis behind explicit tool contracts and source attribution.	P2

ID	Capability	Requirement / expected behavior	Priority
FR-10	Code execution	Any code runner is disabled by default for public deployment. If enabled in a later phase, it must be strongly sandboxed, resource-limited, and user-approved.	P3
FR-11	Settings	Manage model preference, local/cloud availability, memory controls, profile/preferences, and notification settings.	P1
FR-12	Notifications & live state	Provide real-time chat streaming and ingestion/job status updates; send email notifications only for configured user events.	P2
FR-13	Operational logging	Record structured request/job outcomes, latency, tool/model identifiers, and errors without logging secrets or full sensitive content by default.	P0

6. System architecture and layers

The system uses clear service boundaries. The web client communicates with the Node API for product operations. The Node API invokes the Python orchestration service for model/agent tasks, while persistence is split by access pattern: PostgreSQL for canonical records, ChromaDB for vector retrieval, Neo4j for relationship traversal, Redis for ephemeral state, and local/object file storage for source binaries.

Layer	Exact default technology	Responsibilities / contract
1. Client / frontend	Next.js (React, App Router) + TypeScript + Tailwind CSS	Chat, notebook, upload, graph view, workspace, settings; responsive UI; client-side validation; calls API over HTTPS; receives stream/live updates.
2. API gateway	Node.js 22 LTS + Express.js	JWT authentication and authorization, request validation/sanitization, rate limits, routing (/chat, /upload, /notes, /graph, /user), CORS/headers, Winston + Morgan logging, API versioning; REST JSON and Socket.IO/WebSocket endpoints.
3. Backend services	Node.js + Express modules	Chat/session facade, file service, note service, graph service, user/preferences service, notification service; orchestration handoff; job lifecycle; database transactions and ownership checks.

Layer	Exact default technology	Responsibilities / contract
4. AI orchestration	Python 3.12 + FastAPI + LangChain	Intent detection, agent selection, task decomposition, model selection, retrieval/tool routing, tool execution policy, answer synthesis, streaming/result normalization. LangChain is selected from the diagram's LangChain/LlamaIndex alternatives. FastAPI is the proposed Python service interface to make the Python layer deployable independently.
5A. RAG pipeline	Hugging Face Sentence Transformers; ChromaDB	Normalize text, chunk, embed, persist embeddings and metadata, query vector candidates, apply metadata filters, merge context, return source citations. Default embedding: sentence-transformers/all-MiniLM-L6-v2.
5B. Knowledge graph	Neo4j Community Edition + Cypher	Entity/relationship extraction, upsert graph records, maintain source provenance, traverse/query related concepts, serve graph view/query results.
5C. Memory & persistence	PostgreSQL; Redis; local filesystem for MVP	PostgreSQL is canonical relational store; Redis is cache/session/job/stream state only; local filesystem stores files in development/single-host MVP. Object storage is an explicitly configurable later deployment adapter.
6. LLM layer	Ollama local; default model Llama 3 (8B-class); optional Mistral 7B or Gemma. Cloud fallback: Groq API, then Hugging Face Inference API.	Provider adapter selects local default, cloud fallback only when enabled/available. Model timeouts, token/context budgets, provider errors, and cost/use limits are enforced.
7. Real-time & communication	Socket.IO (WebSocket transport); REST for ordinary operations	Chat token/delta events, upload/indexing status, note/graph update notifications, and job progress. Reconnect behavior must recover status from API/job state.
8. External services	YouTube Data API; SerpAPI; Hugging Face Datasets; Firebase Hosting/Storage; Resend	Optional connectors: YouTube metadata/transcript path, web search, dataset access, hosting/storage option, email. Each connector is isolated behind an adapter and key/consent gate.
9. Hosting / runtime	Local machine / workstation; native Node.js and Python processes plus locally installed stateful services.	Default is local-only. Vercel, Railway, Fly.io, and Firebase are pictured optional remote hosting alternatives, not required to run the MVP. No Docker/container images.

6.1 Stack decisions and alternatives

Area	Selected MVP default	Alternatives / handling
Frontend framework	Next.js + React + TypeScript + Tailwind	React may be used through Next.js; do not create a second SPA stack.

Area	Selected MVP default	Alternatives / handling
Node API	Express.js on Node.js 22 LTS	The diagram names Node.js and Express; framework remains Express for consistency.
Python agent interface	Python + LangChain; proposed HTTP adapter: FastAPI	The diagram specifies Python orchestration and LangChain/LlamaIndex. LangChain is selected; LlamaIndex is an alternative, not a second simultaneous framework. FastAPI is an explicit implementation choice not named in the diagram.
Local model	Ollama with Llama 3 8B-class default	Mistral 7B and Gemma are selectable alternatives; model availability depends on host memory/compute.
Cloud model fallback	Disabled by default; optional Hugging Face Inference API or Groq API	Core mode uses local Ollama. Cloud providers are explicitly enabled only by the user; prompts/context leave the machine only after that choice. Provider keys are secrets; free quota/availability is not guaranteed.
Embeddings	Hugging Face Sentence Transformers all-MiniLM-L6-v2	A different Hugging Face embedding model requires re-index/migration plan.
Vector index	ChromaDB	FAISS is an evaluation/local embedded alternative; select one vector persistence backend per environment.
Graph store	Neo4j Community Edition	Cypher query API; Community Edition limits must be checked for hosting topology.
Files	Local filesystem for local/single-host MVP	Firebase Storage is the pictured managed option; abstract storage interface before multi-instance deployment.
Hosting	Local machine is the MVP deployment target	Vercel, Railway, Fly.io, and Firebase Hosting are diagram-listed remote alternatives only. They are not required for local operation. Never deploy as Docker images.

6.2 Complete architecture-diagram traceability

This matrix explicitly accounts for the components and product labels in the supplied architecture image. “Selected” means the MVP default in this PRD; “Alternative” means an either/or option and not a simultaneous requirement; “Optional” means an integration or feature behind a setting/phase gate; “Deferred” means not enabled for MVP.

Diagram item / exact name	PRD treatment	Status / phase
Next.js; React; Tailwind CSS	Next.js with React, TypeScript, and Tailwind is the client stack for Chat, Notebook, File Upload, Graph View, Workspace, and User Settings.	Selected — Phase 1
Node.js; Express	Node.js 22 LTS and Express.js implement the API gateway and backend services.	Selected — Phase 1

Diagram item / exact name	PRD treatment	Status / phase
JWT Based Auth	JWT-backed authentication/authorization; identity provider details remain an open Phase 0/1 decision.	Selected — Phase 1
Rate Limiting / Request Throttling	Rate limits are mandatory at API and provider/tool boundaries. Specific middleware package is an implementation choice.	Selected — Phase 1
Routing: /chat, /upload, /notes, /graph, /user	Versioned API endpoints under `/api/v1` for chat, uploads, notes, graph, and user/profile operations.	Selected — Phase 1
Request Validation / Input Sanitization	Validate all request schemas and sanitize inputs at service boundaries; reject invalid payloads.	Selected — Phase 1
Logging: Winston / Morgan	Winston for structured application logs; Morgan for HTTP request logging, with sensitive values redacted.	Selected — Phase 1
Chat, File, Note, Graph, User, Notification Services	Implement as Node/Express service modules; graph module talks to Neo4j, notification module owns email/live event integration.	Selected — Phases 1–2; notifications Phase 5
Python AI Orchestration Layer	Separate Python service for routing and model/tool orchestration; FastAPI is the proposed transport framework, not a technology named in the diagram.	Selected — Phase 2 onward
LangChain / LlamaIndex	Select LangChain for MVP RAG/orchestration. LlamaIndex is a mutually exclusive alternative to evaluate later; do not run both by default.	LangChain selected; LlamaIndex alternative
Research Agent; Summary Agent; Code Agent; Decision Agent	Bounded agent profiles for web research, summaries, code generation/explanation, and decision support. Code agent explains/generates text only; no host execution.	Phased — Research/Summary Phase 4; Code/Decision guarded
Agent Orchestrator (Router): intent detection, selection, task decomposition	Route intent to a bounded agent/tool plan with a standard-chat fallback and traceable result.	Selected — Phase 4
Web Search; Calculator; File Reader; YouTube Transcript	Tool adapters; SerpAPI powers web search, calculator is local deterministic logic, file reader reads authorized files, and YouTube transcript uses an approved transcript/metadata path.	Optional — Phase 4
Image Analyzer	Image understanding/analysis tool for supported image uploads; provider/model capability and privacy gate required.	Optional — Phase 4
Code Executor	Not enabled in MVP. Any future execution requires a separately approved hardened sandbox, resource limits, and explicit user approval; never run untrusted code directly on host.	Deferred / disabled

Diagram item / exact name	PRD treatment	Status / phase
Key differentiators: Memory Depth; Graph-Based Understanding; Offline Capability; Actionable Outputs; Multi-Modal & Multi-Source; Explain Thinking	Covered by user-controlled memory, Neo4j provenance graph, local Ollama mode, user-approved artifacts, multi-source ingestion, and visible agent/tool/source traces.	Included across Phases 1–4
Prompt-less (Future): AI understands intent, less typing	Future UX direction: infer common intent from context and offer suggested actions while retaining explicit user control; not a Phase 1 requirement.	Future / later
RAG Pipeline: LangChain / LlamaIndex	LangChain chosen for MVP retrieval orchestration; LlamaIndex retained as an alternative, not a parallel dependency.	Selected / alternative
Hugging Face Embeddings; Sentence Transformers	Use Hugging Face Sentence Transformers; initial default model is `sentence-transformers/all-MiniLM-L6-v2`.	Selected — Phase 2
ChromaDB; FAISS	ChromaDB is the persisted vector database default. FAISS is a local/evaluation alternative only; only one vector backend per environment.	ChromaDB selected; FAISS alternative
Neo4j Community Edition	Knowledge graph storage and Cypher query engine.	Selected — Phase 3
PostgreSQL	Canonical relational database for accounts, workspaces, documents, notes, conversations, memory metadata, and jobs.	Selected — Phase 1
Redis	Cache, ephemeral session/job/stream state; not the canonical store for product records.	Selected — Phase 2 as needed
Local Storage; Firebase (Free Tier)	Local filesystem is the default and required for local mode. Firebase Storage/Hosting is an optional remote alternative only.	Local selected; Firebase optional
Ollama; Llama 3 8B; Mistral 7B; Gemma	Ollama is the primary local model runtime. Llama 3 8B-class is the default family; Mistral 7B and Gemma are selectable model alternatives subject to hardware and verified model tags.	Selected local-first — Phase 2
Hugging Face Inference API; Groq	Cloud inference providers, disabled by default. Enable only with explicit user configuration/consent; provider order is configurable.	Optional remote fallback — Phase 2
Try Local LLM; fallback to Cloud API	Attempt local Ollama when configured/available; if unavailable or policy permits fallback, use the configured cloud provider. Report provider used and errors.	Selected behavior — Phase 2
WebSockets; Socket.io	Socket.IO for chat streaming, status and live updates; REST remains for ordinary request/response operations.	Selected — Phase 2
YouTube Data API	Optional YouTube metadata/source integration; transcript availability and separate transcript retrieval path must be validated.	Optional — Phase 4

Diagram item / exact name	PRD treatment	Status / phase
SerpAPI	Optional web-search provider; API key, quotas, and attribution required.	Optional — Phase 4
Hugging Face Datasets	Optional dataset discovery/ingestion connector; not required for users' own-file RAG MVP.	Optional / later
Firebase (Hosting / Storage Free Tier)	Remote hosting/storage alternative shown in the diagram; local filesystem and local frontend are the default. Verify current tier limits if enabled.	Optional remote alternative
Resend	Email provider for configured notifications; never required for core chat/notebook use.	Optional — Phase 5
Vercel; Railway; Fly.io	Exact remote hosting candidates named in the diagram; not needed for local mode. If later used, select per component based on native runtime support and persistence.	Optional remote deployment
Free-first; no Docker	Local/open-source stack is the required default. No Docker/Compose/container deployment; cloud services are optional and are not required for core product operation.	Hard constraint

6.3 Service boundaries and API contracts

- Frontend → Node API: versioned HTTPS JSON API (`/api/v1`), JWT bearer/session authorization, upload multipart endpoint, paginated list endpoints, and Socket.IO connection for live events.
- Node API → Python orchestrator: authenticated internal HTTP JSON requests to `/v1/agent/run` and `/v1/agent/stream`; payload includes user/workspace IDs, conversation ID, prompt, allowed tools, retrieval scope, and request trace ID.
- Python orchestrator → retrieval/tool adapters: explicit interfaces for query, embedding, graph lookup, web search, transcript retrieval, calculator, file reader, and model inference.
- Node services own product authorization and canonical user/workspace operations. Python must not accept untrusted workspace identifiers without an authorization-scoped context minted by Node.
- No service may expose PostgreSQL, Redis, ChromaDB, Neo4j, or local files directly to the browser.

6.4 Core data model

Entity	Canonical location	Key fields / relationships
User	PostgreSQL	id, email/identity provider reference, profile, preferences, created_at.
Workspace	PostgreSQL	id, owner_user_id, name, settings; all user data is scoped through workspace_id.
Document / Asset	PostgreSQL + file storage	id, workspace_id, owner_id, filename, MIME, storage key/path, hash, size, status, metadata, timestamps.
DocumentChunk	PostgreSQL metadata + ChromaDB vector	chunk_id, document_id, workspace_id, ordinal, text reference, embedding ID, page/section metadata.

Entity	Canonical location	Key fields / relationships
Conversation / Message	PostgreSQL	conversation_id, workspace_id, role, content, model/provider, state, citations/tool trace, timestamps.
Note	PostgreSQL	note_id, workspace_id, title, body, linked documents/entities/conversation IDs, timestamps.
MemoryItem	PostgreSQL + ChromaDB optional vector	user/workspace scope, type, content, source, confidence, sensitivity flag, approval/deletion metadata.
Graph entity/relation	Neo4j + relational provenance	workspace-scoped entity nodes and typed relationships; source IDs, extraction provenance, confidence, created/updated times.
Job / audit event	PostgreSQL; Redis transient status	job ID, type, status, progress, trace ID, outcome/error; minimal security/operational event records.

6.5 Key data flows

1. Document ingestion: upload/link → authenticate and validate → store source asset → extract text/metadata → normalize and chunk → compute embeddings → persist vectors and chunk metadata → extract entities/relations when enabled → upsert Neo4j graph → mark indexing status and notify client.
2. Chat/query: authenticate → resolve workspace scope → detect intent → retrieve conversation context and user-approved memory → query ChromaDB and optionally Neo4j → select model/provider → generate grounded response with citations → persist message and trace → stream tokens/result over Socket.IO.
3. Memory update: after a conversation, propose candidate facts/preferences → apply sensitivity and relevance policy → require user consent/configured policy → persist approved memory → update relevant vector/graph indexes → expose view/edit/delete controls.
4. Agent task: classify intent → select bounded agent → produce plan → obtain required user approval for consequential actions → invoke allowlisted tools → collect evidence → synthesize response and source trail → persist task outcome.

6.6 Repository structure and file ownership

Use a single repository with strict folder ownership. Frontend code belongs under `apps/web`; the Node product backend belongs under `services/api`; the Python model/RAG/agent backend belongs under `services/ai`. Keep shared schemas in `packages/contracts`, project documentation in `docs`, setup/run/backup scripts in `scripts`, and runtime data in gitignored `data-local`. Do not scatter files at the repository root or create duplicate frontend/backend folders.

```
neurodesk-ai/
├── AGENTS.md                # Antigravity persistent project rules
├── README.md                # Local setup, run, test, backup, troubleshooting
├── package.json             # Root scripts only
├── pnpm-workspace.yaml      # JavaScript workspace declaration
├── .env.example             # Safe placeholders only; no secrets
├── apps/web/                # Next.js + React + TypeScript + Tailwind UI
│   ├── app/                 # Routes and layouts
│   ├── components/          # Shared visual components
│   ├── features/            # chat/, notebook/, files/, graph/, memory/, settings/
│   ├── hooks/               # React hooks
│   ├── lib/                 # Browser-safe API client/helpers
│   └── tests/               # Frontend tests
├── services/api/            # Node.js + Express product API
│   ├── src/routes/          # HTTP endpoints
│   ├── src/services/        # Business logic
│   ├── src/middleware/      # Auth, validation, rate limits, logging
│   ├── src/db/              # PostgreSQL access and migrations
```

```
│   └── tests/                # API tests
├── services/ai/              # Python + FastAPI + LangChain
│   ├── app/agents/            # Research, summary, code-assist, decision
│   ├── app/providers/         # Ollama and optional cloud adapters
│   ├── app/retrieval/         # Embeddings, ChromaDB, RAG
│   ├── app/graph/             # Neo4j access
│   ├── app/tools/             # Search, calculator, file, transcript, image
│   └── tests/                 # AI service tests
├── packages/contracts/        # OpenAPI, JSON schemas, generated shared types
├── docs/                      # PRD, UI reference, UI direction prompt, decisions
├── scripts/                   # Native local setup/start/stop/backup scripts
└── data-local/                # Gitignored uploads/, chroma/, backups/
```

- Frontend-only components, client state, and browser UI stay in `apps/web`; server routes, auth, ownership checks, and product business logic stay in `services/api`; prompts, model routing, retrieval, embeddings, graph access, and AI tools stay in `services/ai`.
- Tests mirror their owning service under its `tests/` directory. PostgreSQL migrations live with the Node API database package. Shared request/response schemas live once in `packages/contracts` and are consumed by both API and UI.
- Before adding a folder, inspect the existing tree and reuse its naming. Do not move unrelated files. Keep root files limited to workspace configuration, README, agent rules, and shared developer configuration.
- Keep `.env`, database files, uploads, ChromaDB persistence, model caches, logs, and backups out of Git. Commit only templates and scripts. Never commit passwords, API keys, or real user data.

6.7 UI example and screen behavior

The default screen is a local-first research workspace with persistent navigation, a central task surface, and a contextual source panel. The standalone `docs/UI_REFERENCE.md` is the visual handoff to frontend implementation; `docs/UI_DIRECTION_PROMPT.md` is the paste-ready Antigravity instruction. Both retain the existing NeuroDesk palette: navy #152B50, blue #1769AA, teal #16856B, pale #EEF4FA, gray #586579, and sparing gold #D3AA27. Do not replace or extend the brand palette with colors from inspiration sites.

Screen region	Example content and behavior
Workspace home hero	Compact in-app welcome/task headline; one-sentence context; Start a Chat, Add Sources, and Open Graph actions. Beneath it, show only real source/note/concept counts, recent source cards, recent activity rows, and an optional graph preview; hide fabricated or empty metrics.
Top bar	NeuroDesk AI brand; workspace search; New Chat action; visible `Local mode · Ollama` status badge.
Left navigation	Chat, Notebook, Files, Knowledge Graph; collection list; Settings and local service status near the bottom.
Main content — empty chat	Prompt such as “What would you like to understand?”; quick actions for summarizing sources, comparing ideas, creating study notes, or exploring a topic.
Main content — active chat	Readable user/assistant turns; streamed response; compact source citations such as `[Reading.pdf · p. 4]`; retrieval/tool activity status; useful no-evidence message.
Persistent composer	Question input; attach-file action; workspace scope selector; Send action; local model/provider label.
Right context panel	Upload dropzone; current source list with processing status; Ollama/model status; cloud inference shown Off unless explicitly configured.
Responsive behavior	On narrow screens, collapse navigation and source panel into drawers/tabs; preserve a reachable composer and readable citations without horizontal scrolling.

Implement the shell as reusable components (AppShell, WorkspaceSidebar, WorkspaceHeader, MainContent, ContextPanel, ChatComposer). Route-specific views live under `apps/web/features/` and must not become one giant page component. Maintain keyboard focus, semantic headings, readable contrast, and a text/list alternative for the graph canvas.

Translate selected inspiration features into product UI: portfolio cards become recent-source cards; service rows become agent/tool or indexing-job rows; progress meters report real ingestion/task progress; the technical-map treatment may inspire a labeled knowledge graph; and concise pill actions may become quick actions. Use ordinary lists instead of carousels unless a small, genuinely ordered set benefits from one. Do not add testimonials, partner logos, marketing claims, or decorative contact calls to action.

Supplied website prompt	Selected product adaptation	Explicitly excluded
Laocoön — Bronze and Time	Editorial content hierarchy, fine alignment grid/dividers, room for a featured answer or notebook, and lightweight progress feedback.	Bronze/sapphire palette, statue imagery, WebGL/particles, custom cursor, long fixed scroll narrative.
Lumora — Design & Engineering Studio	Responsive spacing, modular cards/chips, grouped content, selective contrast, restrained hover/focus feedback.	Burnt orange palette, agency/portfolio marketing sections, pointer-painted reveal, loader-gated access.
KIMI / GRIDO1 Racing Systems	Compact identity/metadata rows and a technical-map visual idea for knowledge graph exploration.	Racing branding, cyan palette, pinned/receding stack, F1 imagery, heavy circuit/WebGL spectacle.
Baseline — Tennis Club & Academy	Heading + context + primary-action hierarchy, scannable rows, useful card groups, accessible dialog mechanics.	Tennis brand, marketing section sequence, global smooth-scroll hijacking, loader that blocks the app.
Soda	One clear focal surface and subtle layering for contextual panels where contrast remains strong.	Beverage imagery, pink/teal gradients, 3D can/particles, flavor carousel, fixed no-scroll layout.

The inspiration set contains five website recreation briefs in `website_recreation_prompts_144_pages_fixed.pdf`. Their patterns are selectively adapted above; they do not override the product requirements, local-first behavior, existing brand palette, accessibility requirements, or Next.js/React/Tailwind stack.

7. Security, privacy, and trust requirements

- All APIs enforce workspace ownership on every read/write; use deny-by-default authorization and regression tests for cross-workspace access.
- Transport uses HTTPS in hosted deployments; secrets are environment variables/provider secret stores and never committed to source control.
- Validate file extension, MIME signature, size, and processing limits; scan or safely reject unsupported/malformed files; do not execute uploaded content.
- Use parameterized database operations, secure password/session practices, JWT expiry/rotation strategy, CSRF controls where cookie sessions are used, and strict CORS allowlists.
- Avoid logging raw prompts, file contents, access tokens, or model secrets by default. Provide configurable retention and deletion workflows.
- User can view/delete uploaded files, notes, conversations, and memory. Deletion must enqueue cleanup from PostgreSQL, file storage, ChromaDB, and Neo4j with retryable status.
- Cloud inference is disabled unless the user/admin configures it; disclose that prompts/context are sent to the selected provider when enabled.
- Web search, YouTube, and file-derived text are untrusted input. Defend against prompt injection by treating content as evidence, not policy; tools use allowlists and bounded arguments.
- Code execution is excluded from MVP and remains disabled unless a separate sandbox design is approved. No Docker does not mean untrusted code may run on the host.

8. Non-functional requirements

Category	Requirement / initial target
Availability	Best-effort pilot availability; graceful degradation when any optional provider is unavailable. Local development remains usable without cloud model keys.
Performance	Interactive API operations p95 under 500 ms excluding model generation and large uploads. First streaming model output target under 8 seconds for normal requests on supported hardware/provider; measure rather than guarantee.
Reliability	Ingestion is asynchronous and idempotent by asset hash/job key; retries use bounded backoff; failed jobs surface actionable errors and can be retried.
Scalability	Stateless Node/Python web processes can scale horizontally only after moving files/session/job state to shared managed services and validating graph/vector storage concurrency.
Accessibility	Aim for WCAG 2.2 AA for core navigation, chat, forms, status, and graph alternatives; keyboard-accessible graph list view is required.
Observability	Structured logs with trace IDs, service health endpoints, latency/error metrics, job status, model/provider usage counters, and redacted diagnostics.
Portability	Document native installation, environment variables, migrations, model download/setup, backup/restore, and service startup without Docker.
Cost control	Rate limits and quotas for provider calls, file size/page limits, model selection policy, and visibility into cloud fallback use.

9. Phased delivery plan

Phases are dependency-ordered delivery increments. Calendar estimates are intentionally omitted until team size, hosting constraints, and model hardware are confirmed. Each phase is complete only when its exit gate passes; later-phase scope must not block the earlier usable product.

Phase 0 — Product & engineering foundation

Scope: Define user journeys, supported file types/limits, data retention, model fallback consent, and deployment target. Create repository, branching/check pipeline, environment configuration, TypeScript/Python code conventions, service contracts, architecture decision records, and native setup scripts.

Exit gate: Repository builds locally without Docker; Next.js, Express, and FastAPI skeletons start as separate native processes; health checks pass; secret and environment configuration documented; API contract and data ownership rules reviewed.

Phase 1 — Workspace, identity, and notebook shell

Scope: Build frontend navigation for Chat, Notebook, Files, Graph, Workspace, and Settings. Implement auth/session, profile/preferences, workspace isolation, PostgreSQL migrations, basic note CRUD, upload UI and API, validation, file metadata, and operational logging.

Exit gate: User can sign in, create/read/update/delete a note, upload an allowed file, view processing state, and cannot access another workspace's data; deployable without Docker.

Phase 2 — Document ingestion and grounded RAG

Scope: Implement extraction, normalization, chunking, Hugging Face Sentence Transformers embedding, ChromaDB persistence, search/filtering, citations, file reader, and grounded chat using Ollama. Implement provider adapter and opt-in Groq/HF fallback.

Exit gate: Given a known test corpus, retrieval returns relevant chunks; answers cite correct source/page/section; unsupported files fail safely; local Ollama path works with no cloud key; provider outage is handled.

Phase 3 — Durable memory and knowledge graph

Scope: Add user-controlled memory proposals, review/edit/delete UX, memory retrieval, entity/relationship extraction, Neo4j upsert/query, graph view, graph provenance, and deletion propagation across stores.

Exit gate: User can inspect and remove memory; graph nodes/edges link to source documents; query answers can use relevant graph context; deleted source/memory is removed or explicitly tracked as pending cleanup across all indexes.

Phase 4 — Multi-agent orchestration and tools

Scope: Add LangChain-based router, intent classification, agent registry (research, summary, code-assist explanation, decision support), task decomposition, allowlisted web search/calculator/transcript/image-analysis/file-reader tools, Socket.IO streaming, and traceable tool results.

Exit gate: Agent routing is deterministic for test cases; each answer exposes agent/tool/source trace; tools honor permissions and rate limits; no unapproved external or consequential action occurs.

Phase 5 — Hardening, launch readiness, and integrations

Scope: Add load/security tests, retention/export/delete flows, local backups/restore, monitoring, deployment/startup scripts for native local processes, resilience, and runbooks. Add Resend, YouTube Data API, SerpAPI, Hugging Face Datasets, cloud inference, or remote hosting only as separately configured optional integrations.

Exit gate: Security and cross-workspace tests pass; local restore drill succeeds; operator can start/stop/recover every core local service without Docker or cloud credentials; optional providers fail gracefully; privacy disclosures are in place.

10. Release acceptance criteria

- A new user can authenticate and reach an isolated workspace with Chat, Notebook, Files, Graph, and Settings navigation.
- A user can upload a supported document and observe validation, ingestion status, and a useful failure reason if processing fails.
- A question over indexed source material returns an answer with source references; answers without retrieved evidence clearly state uncertainty rather than inventing citations.
- Notes, conversations, user preferences, and approved memory persist after service restart.
- Graph view can render source-linked nodes/edges, and a text/list alternative is available.
- The full core product works in local-only mode with Ollama and no cloud inference key; cloud inference is disabled by default and only used after explicit configuration.
- Core data operations enforce workspace authorization and pass cross-tenant negative tests.
- System can be set up, run, test, back up, and recover using locally running native Node/Python/database/model processes only; no cloud account, Docker files, or container runtime is required for core operation.

11. Risks, assumptions, and mitigations

Risk / assumption	Impact	Mitigation / decision
Free-tier providers change limits or remove persistence	Unexpected cost, data loss, or downtime	Treat free-tier as an initial target only; verify current terms before launch; keep providers behind adapters; define backup/export.
Local LLM hardware varies significantly	Slow inference or model cannot load	Document minimum hardware; use a smaller configurable Ollama model; allow opt-in remote fallback; show model availability status.
Multiple stores can become inconsistent	Stale vectors/graph after deletes or re-index	Use durable ingestion/deletion jobs, idempotency, reconciliation reports, and explicit pending-cleanup state.
Cross-store semantic/graph behavior is complex	Incorrect or confusing answers	Keep provenance IDs, citations, confidence, source filters, and regression test corpus; graph context supplements rather than replaces source evidence.
Provider APIs and rate limits vary	Tool/model failure	Provider adapters, timeouts, bounded retries, per-provider quotas, graceful degradation and test doubles.
No Docker increases setup variability	Onboarding/deployment drift	Pin runtimes, provide OS-specific native setup scripts, lockfiles, service files/systemd examples, health checks, and reproducible migration steps.
Free plan may not support durable multi-service production	Production reliability unclear	Pilot first; choose managed database/storage or paid runtime if required; do not promise production-grade zero-cost hosting.

12. Open decisions before Phase 1

- Authentication method: email/password, OAuth provider, or both; identity provider not specified in the architecture diagram.
- Supported file formats, per-file limits, OCR provider/engine for image-only PDFs, and retention policy.
- Exact primary local model artifact/quantization and minimum workstation/server memory/compute.
- Local workstation/OS targets and minimum resource requirements for PostgreSQL, Redis, Neo4j, Ollama, and the app processes; optional remote hosting is out of the local MVP path.
- Whether graph construction and external web/video tools are on by default or enabled per workspace.
- Definition of sensitive data categories excluded from automatic memory capture.
- Expected active users and usage volume to set rate limits, load targets, and spend ceilings.
- Whether any optional remote hosting/storage adapter (Firebase, Vercel, Railway, or Fly.io) is needed after local MVP validation.

Appendix A — Runtime and deployment model (no Docker)

Default runtime is one local workstation or self-managed local server, with each service installed and run as a native process—never in Docker: Next.js development/production server; Node/Express API; Python/FastAPI orchestrator; PostgreSQL; Redis; Neo4j Community; ChromaDB persistent local storage; Ollama; and local file storage. Use package lockfiles (pnpm for JavaScript; pinned Python dependencies), environment variables, native installers, database migrations, and local backup/restore scripts. Core workflows must work with internet access disabled after software/model setup; internet-only connectors are visibly unavailable in that mode.

Local service	Default local endpoint / storage	Startup dependency
PostgreSQL	127.0.0.1:5432	Start first; apply migrations.
Redis	127.0.0.1:6379	Start before API/AI services that use cache or job state.
Neo4j Community	Bolt 127.0.0.1:7687; Browser 127.0.0.1:7474	Start before graph service operations; local credentials stored outside source control.
Ollama	127.0.0.1:11434	Install and pull a compatible local model; default local inference provider.
ChromaDB	Embedded persistent mode; configurable local data directory	No separate daemon required in default mode; local persistent path is backed up.
Python AI service (FastAPI)	127.0.0.1:8000	Start after local stores/model runtime are available.
Node API (Express)	127.0.0.1:4000	Start after PostgreSQL and required local dependencies.
Next.js web client	127.0.0.1:3000	Start after API is reachable; browser opens local URL.
Local file storage	Configured directory under local application data	Create with restrictive permissions; never place user data in ephemeral build output.

Recommended startup sequence: install runtimes and pull the selected Ollama model → start PostgreSQL, Redis, and Neo4j → run migrations and verify ChromaDB local persistence → start Python AI service → start Node API → start Next.js. Stop in reverse order. Provide health checks, a local environment template, service logs, secrets guidance, local backup/restore and recovery instructions. Default network binding should be localhost; expose to a LAN only with explicit configuration and authentication.

Remote hosting is not the default runtime. Vercel, Railway, Fly.io, and Firebase are retained only because they appear in the supplied architecture image; using them is a separate optional deployment decision and is not necessary for the local MVP. Hugging Face Inference API, Groq, SerpAPI, YouTube Data API, Hugging Face Datasets, Firebase, and Resend are external integrations and remain disabled/unconfigured in strict local-only mode. Never rely on ephemeral remote disks for user data if a remote deployment is later approved.

Appendix B — Recommended engineering practices

- Use the repository tree and file ownership rules in Section 6.6; store native local setup/start/stop/backup scripts under `scripts/`, not a Docker or container directory. Read `docs/UI_REFERENCE.md` and `docs/UI_DIRECTION_PROMPT.md` before frontend work.
- Use OpenAPI for public/internal HTTP contracts and schema validation at boundaries; version API changes under `/api/v1`.
- Use database migration tooling for PostgreSQL and versioned ingestion metadata; keep vector and graph IDs traceable to canonical relational IDs.
- Automated tests: unit tests for parsers/router/policies; integration tests for database/retrieval adapters; end-to-end tests for upload → indexed answer → citations; authorization regression tests.
- Use feature flags for optional cloud inference, graph extraction, external tools, and code-assist; code execution remains disabled.
- Record architecture decisions on model/provider choice, storage provider, auth, OCR, and deletion semantics before these become difficult to change.

NeuroDesk AI — All-in-One Project Handoff

This single PDF is the consolidated project handoff. It contains the phased PRD, exact technology stack, local/no-Docker requirements, Antigravity workspace rules, UI reference, and paste-ready UI direction prompt.

Use with Antigravity

1. Keep this PDF with the project, for example at `docs/NEURODESK_AI_COMPLETE_PROJECT_PACK.pdf`.
2. Ask Antigravity to read the whole PDF. Treat the PRD as the product source of truth, follow the workspace rules, and preserve the specified UI palette.
3. Start with Phase 0 only. Do not use Docker. Keep frontend, API, AI service, shared contracts, documentation, and local data in their assigned folders.
4. Before each later phase, review the current implementation and test results, then request that phase only.

Contents in this PDF

- **Part 1 — Product Requirements Document:** features, exact technology stack, architecture layers, requirements, phased delivery plan, acceptance criteria, local runtime, and security.
- **Part 2 — Antigravity Setup and Workspace Rules:** instructions for using the document, project layout, file boundaries, local services, no-Docker invariant, and implementation checks.
- **Part 3 — UI Reference:** preserved NeuroDesk palette, screen behavior, feature/component patterns, and selective inspiration map for the five supplied website prompts.
- **Part 4 — Antigravity UI Direction Prompt:** a prompt to paste into Antigravity when implementing the frontend.

The five website prompts inform visual hierarchy, spacing, modular cards, useful status/progress indicators, and graph-map styling only. Their unrelated branding, palettes, heavy effects, and marketing-page layouts are not copied.

NeuroDesk AI — Antigravity Workspace Rules

Keep this repository organized, local-first, and easy to maintain. Follow the latest PRD in `docs/` as the product source of truth and read both `docs/UI_REFERENCE.md` and `docs/UI_DIRECTION_PROMPT.md` before building or changing screens. This `AGENTS.md` belongs at the repository root so Antigravity can discover it as a workspace rule (official Rules documentation).

Project layout — keep code in its assigned area

```
neurodesk-ai/
├── AGENTS.md
├── README.md
├── package.json
├── pnpm-workspace.yaml
├── .env.example
├── apps/web/
│   ├── app/
│   ├── components/
│   ├── features/{chat,notebook,files,graph,memory,settings}/
│   ├── hooks/
│   ├── lib/
│   ├── public/
│   └── tests/
├── services/
│   ├── api/src/{routes,services,middleware,db}/ + tests/
│   └── ai/app/{agents,providers,retrieval,graph,tools}/ + tests/
├── packages/contracts/
├── docs/{PRD.pdf,UI_REFERENCE.md,UI_DIRECTION_PROMPT.md,decisions}/
├── scripts/
└── data-local/{uploads,chroma,backups}/
```

File placement and change rules

1. Put browser UI, client-side state, and visual components only in `apps/web/`. Do not put React components in `services/`.

2. Put authentication, product APIs, ownership checks, PostgreSQL transactions, and business rules only in `services/api/`.
3. Put prompts, model routing, LangChain agents, retrieval, embeddings, Neo4j operations, and AI tools only in `services/ai/`.
4. Put shared API schemas/contracts in `packages/contracts/`. Do not copy/paste duplicate request/response types between frontend and backend.
5. Put migrations with the owning database service; put tests beside their owning app/service under its `tests/` directory. Put docs in `docs/`, scripts in `scripts/`, and local mutable data in ignored `data-local/`.
6. Keep the repository root limited to workspace configuration, README, this rule file, and shared developer configuration. Do not add loose application source files at the root.
7. Before creating a folder or file, inspect the existing tree and reuse its established naming. Do not create duplicate `frontend`, `client`, `backend`, or `server` folders.
8. Make the smallest scoped change that completes the request. Do not move or rename unrelated files. Update the relevant tests and documentation when a contract changes.
9. Keep secrets out of source control. Add new environment variables to `.env.example` with safe placeholders and explain them in README.

Runtime and architecture invariants

- Core development and usage must run locally as native processes. **Do not create Dockerfiles, docker-compose files, container manifests, or Docker-based instructions.**
- Local endpoints: web `127.0.0.1:3000`; Node API `127.0.0.1:4000`; Python AI service `127.0.0.1:8000`; PostgreSQL `127.0.0.1:5432`; Redis `127.0.0.1:6379`; Neo4j Bolt `127.0.0.1:7687` and Browser `127.0.0.1:7474`; Ollama `127.0.0.1:11434`.
- Use Ollama as the default local inference provider. Hugging Face Inference API and Groq are optional cloud providers and must remain disabled unless the user explicitly configures them.
- PostgreSQL is the canonical relational store. Redis is ephemeral cache/job/session state. ChromaDB is the default local vector store. Neo4j Community is the graph store. Local filesystem storage is the default file store.
- Enforce workspace authorization in the Node API for every user-data operation. The AI service must receive only authorized, scoped requests from the API.

- Keep external connectors (SerpAPI, YouTube Data API, Hugging Face Datasets, Firebase, Resend) behind adapters and disabled by default. Core notebook, local RAG, graph, memory, and chat must not require these services.
- Code execution is disabled. Never execute model-generated code or uploaded content on the host.
- Preserve accessibility, responsive behavior, visible citations/source evidence, and explicit local/cloud model status.

UI implementation rule

Build the shell and screen patterns in `docs/UI_REFERENCE.md`, following `docs/UI_DIRECTION_PROMPT.md`. Keep navigation and shared controls in reusable components; keep feature-specific screens in `apps/web/features/`. Use the reference for layout and hierarchy, not as permission to put all UI in one giant page. Preserve the existing brand palette. The default status must say **Local mode · Ollama** when local inference is active. Show source citations with assistant answers and provide a readable list alternative for graph visualization.

Before finishing a task

- Run relevant frontend/API/AI tests and lint/type checks, or clearly report which checks could not be run.
- Confirm changed files remain inside the correct folders and no secret/local database/file data was added to Git.
- Verify the app still starts locally without cloud keys and without Docker when the change touches startup or configuration.
- Summarize what changed, list tests run, and call out any remaining decision or limitation.

NeuroDesk AI — UI Reference for Building

Use this document with `UI_DIRECTION_PROMPT.md` and the PRD when building `apps/web/`. This is a functional, local-first knowledge-work interface—not a recreation of any reference website. Keep the established NeuroDesk shell and palette unchanged.

Design direction

Create a calm, editorial productivity workspace with clear typographic hierarchy, generous but practical spacing, fine dividers, restrained reusable cards, and one focused task/content surface. Preserve the persistent workspace sidebar, top bar, central feature surface, and contextual source panel. Make citations, local-model state, and privacy status visible. Polish should improve orientation and task clarity, not compete with source material.

Existing NeuroDesk palette — preserve exactly

Token	Value	Use
Navy	`#152B50`	Navigation structure, headings, primary text on light backgrounds
Blue	`#1769AA`	Primary actions, links, selected controls
Teal	`#16856B`	Healthy local-service status and secondary semantic emphasis
Pale surface	`#EEF4FA`	Quiet panels, hover surfaces, cards, source context
Gray	`#586579`	Secondary labels, metadata, supporting text
Gold	`#D3AA27`	Rare highlight only; never a broad surface or competing accent
White	`#FFFFFF`	Reading canvas and high-contrast content surfaces

Do not replace these tokens with the five reference sites' black/bronze, burnt-orange, cyan/black, tennis-club navy scheme, or teal/pink gradients. Do not add new brand colors. Use red only for conventional destructive/error semantics if required, with accessible contrast and no effect on the brand palette.

Typography and surface language

Use a readable sans-serif system stack for product UI (for example `Inter, system-ui, sans-serif`). If Onest is selected, bundle/self-host it so local/offline use does not depend on Google Fonts. Reserve display weight/size for page or workspace titles; keep buttons, navigation, labels, citations, and metadata compact and readable. Do not copy the source prompts' oversized all-caps editorial faces, cursive display face, or dense all-uppercase racing style.

Use light canvas surfaces with navy structure, blue/teal interactive states, and pale contextual panels. Dark navy may be used as a compact header, status panel, or focused emphasis—not as an entirely separate competing theme. Use borders and spacing before shadows. Avoid translucent glass if it reduces legibility.

Example: desktop Chat workspace

Keep the desktop workspace in four stable regions:

- **Top bar:** NeuroDesk AI, workspace search, New Chat, and `Local mode · Ollama` status.
- **Sidebar:** Chat, Notebook, Files, Knowledge Graph, collections, Settings, and local health.
- **Central workspace:** task heading, suggested actions, recent-source cards or conversation, and a persistent composer when in Chat.
- **Context panel:** file dropzone, current source list, processing status, local model details, and cloud inference status (Off by default).

Suggested empty-chat actions: Summarize sources, Compare ideas, Create study notes, and Explore a topic. A file card can show its title, file type, page count, and indexing state.

Example: workspace home hero and overview

Use a compact "hero" inside the app's central column—not a separate full-screen marketing page. Give the user one clear focal message and immediate actions. Below it, use real workspace data to create useful source cards, activity rows, and compact counts.

- **Hero content:** a short welcome/task headline, one sentence of context, and Start a chat, Add sources, and Open knowledge graph actions.
- **Summary strip:** real counts for sources, notes, concepts, plus local model health. Hide counts or use an empty state when no data exists.

- **Recent work:** recent-source cards and recent-activity rows, using truthful file titles and actual status.
- **Knowledge context:** a static or lightly interactive related-concepts preview; no continuously animated 3D scene.

Counts and activity must come from real workspace data. Hide or use an empty-state message when there is no data; never show fabricated metrics. The related-concepts preview is a static or lightly interactive summary, not a continuously animated 3D scene.

Example: grounded answer state

The center becomes a readable conversation. An answer can show a small activity line such as `Searching your workspace · 3 relevant passages`, then cite sources with actionable chips such as `[Reading.pdf · p. 4]`. Selecting a citation opens its file or extracted passage in the source panel. If retrieval finds no supporting evidence, say so instead of inventing citations. When the model is running locally, keep `Local mode · Ollama` visible. If optional cloud inference is explicitly enabled, identify the selected provider and indicate that context may be sent off-device.

Main screen patterns

- **Chat:** readable history, a persistent composer, attachment/scope controls, visible source citations, local-model status, and honest no-evidence/error states.
- **Workspace home:** compact hero message with clear actions, a real-count summary strip, recent source cards, recent activity rows, and an optional graph preview. Treat these as task entry points, not promotional content.
- **Notebook:** scannable note list, focused editor, linked source/memory panel, clear saved/unsaved state, and safe create/delete actions.
- **Files:** searchable files with type, indexing state, date, retry/remove actions, progress, and an accessible upload/drop zone.
- **Knowledge Graph:** labeled graph view with search/filter, selected-node details and citations in the context panel, plus keyboard-accessible entity/relation list.
- **Memory:** user-visible proposed/saved memory, source/provenance, edit/delete controls, and clear consent/settings.
- **Settings:** profile/preferences, local service health, local model choice, memory/privacy controls, and optional provider setup; cloud stays Off by default.

Translate the reference-site components into product components only where useful: portfolio cards become recent-source cards; service rows become agent/tool or indexing-job rows; a progress meter becomes document-ingestion or task progress; a technical map becomes the knowledge-graph canvas; a carousel becomes an explicit model/view selector only if there are a few genuine options. Do not add testimonials, partner logos, marketing stats, or decorative contact CTAs.

Use carousels only when users need to browse genuinely ordered items. Otherwise use an ordinary list or card grid. Use dialogs for focused operations such as source import only; include labels, visible close action, keyboard focus management, Escape behavior, and restored focus.

Component starting point

```
AppShell
├── WorkspaceSidebar
│   ├── WorkspaceSwitcher
│   ├── PrimaryNavigation
│   ├── CollectionList
│   └── LocalStatusBadge
├── WorkspaceHeader
│   ├── GlobalSearch
│   ├── NewChatButton
│   └── ModelStatus
├── MainContent
│   ├── ChatView | NotebookView | FilesView | GraphView | SettingsView
│   └── ChatComposer (chat route only)
└── ContextPanel
    ├── UploadDropzone
    ├── SourceList
    ├── CitationDetail
    └── LocalModelCard
```

Selective inspiration from the five supplied website prompts

The attached PDF contains these five recreation briefs. Reuse only these compatible principles; do not reproduce their pages or branding.

Reference prompt	Adapt into NeuroDesk	Do not copy
Laocoön — Bronze and Time	Editorial content hierarchy, fine alignment grid/dividers, small progress feedback, room around a featured note or answer	Bronze/sapphire palette, statue imagery, WebGL, particles, custom cursor, long fixed scroll narrative
Lumora — Design & Engineering Studio	Responsive spacing, modular cards and chips, grouped content, selective light/dark contrast, calm hover/focus behavior	Burnt orange, agency/portfolio marketing sections, pointer-painted image reveal, loader-gated access
KIMI / GRIDO1 Racing Systems	Compact identity/metadata rows, strong section labels, optional technical-map feel for graph visualization	Cyan/black racing palette, F1 content, pinned/receding page stack, elaborate WebGL/circuit effects
Baseline — Tennis Club & Academy	Clear heading + context + primary action, ordered rows, useful card groupings, accessible dialog patterns	Tennis branding, testimonials/programs landing page sequence, global smooth-scroll/loader behavior
Soda	One clear focal surface, subtle layering for context cards, concise secondary panel	Pink/teal gradients, beverage imagery, floating 3D objects, flavor carousel, no-scroll layout

Motion, interaction, and accessibility rules

- Keep motion brief, optional, and useful for selection, content arrival, indexing, or panel transitions. Never delay access to core content behind an intro loader.
- Respect `prefers-reduced-motion`; give every animated state a static equivalent. Do not tie reading, navigation, graph use, or task completion to scroll position or pointer motion.
- Keep native scrolling, cursor, text selection, and scrollbars. Do not add scroll hijacking, forced cursors, global scroll locking, or full-page pinned stacks.
- Use fine lines and decorative details as pointer-transparent layers. Do not obstruct selecting text, links, controls, or graph nodes.

- Provide keyboard and touch operation, visible `:focus-visible` states, semantic headings/status, accessible names, readable contrast, and a text/list alternative for graph content.
- Reflow sidebars/panels into drawers or tabs on small screens. Keep the composer reachable and citations readable without horizontal scrolling.

Component ownership

The app shell and shared visual elements live in `apps/web/components/`. Feature views and their local components live under `apps/web/features/chat/`, `notebook/`, `files/`, `graph/`, `memory/`, and `settings/`. Keep routes/layouts in `apps/web/app/`; shared browser-safe tokens/API helpers in `apps/web/lib/`. Do not put React UI in either backend service. See `AGENTS.md` for the full folder ownership tree and local runtime rules.

Antigravity UI Direction Prompt — NeuroDesk AI

Paste the following instruction into Antigravity when planning or implementing the NeuroDesk frontend.

You are building NeuroDesk AI, a functional local-first AI notebook, chat, memory, file library, and knowledge-graph workspace. Read `AGENTS.md`, the PRD, the UI reference, and this direction prompt before changing the UI. Work only in the PRD-approved scope and keep all frontend code under `apps/web/`. Do not implement backend code in this task unless explicitly asked.

Design goal

Build a calm, editorial productivity workspace with clear hierarchy, practical whitespace, fine dividers, reusable cards/chips, compact metadata, and one focused task/content surface. This is a working app—not a marketing landing page and not a recreation of any source website. Preserve the existing shell: stable workspace sidebar, top bar, central feature surface, and contextual source panel. Keep citations, local-model status, and privacy cues visible.

Keep this palette exactly

- Navy `#152B50`: structural navigation and primary text.
- Blue `#1769AA`: primary actions, links, selected controls.
- Teal `#16856B`: healthy local-service state and secondary semantic emphasis.
- Pale `#EEF4FA`: quiet panels, contextual cards, subtle hover surfaces.
- Gray `#586579`: metadata and secondary text.
- Gold `#D3AA27`: rare highlight only.
- White `#FFFFFF`: reading canvas.

Do not add or substitute brand colors. In particular, do not import the references' black/bronze/sapphire, burnt-orange, cyan/black, tennis-brand, dark teal gradient, or pink palettes. Error/destructive states may use conventional accessible semantic color only when needed, not as a brand accent.

Use the five recreation briefs selectively

1. **Laocoön:** use its editorial content hierarchy, alignment grid, thin dividers, and quiet progress feedback. Do not use its cinematic statue scene, WebGL, particles, custom cursor, or scroll-bound narrative.
2. **Lumora:** use responsive spacing, modular cards/chips, grouped content, and selective contrast. Do not copy its agency/portfolio landing page, burnt-orange brand, pointer-painted photo reveal, or loader-gated access.
3. **KIMI / GRIDO1:** use compact identity/metadata rows and the idea of a technical-map visualization for knowledge-graph exploration. Do not copy racing branding, cyan palette, pinned page stacks, circuit spectacle, or WebGL effects.
4. **Baseline:** use the concise heading + context + primary-action hierarchy, scannable ordered rows, useful card grouping, and accessible dialog patterns. Do not copy the tennis marketing section sequence, testimonials, or global smooth-scroll behavior.
5. **Soda:** use one clear focal surface and subtle layering for secondary context panels only. Do not copy beverage imagery, 3D objects, flavor carousel, pink accents, or no-scroll layout.

Screen direction

- **Workspace home:** use a compact in-app hero (not a full-page marketing hero) with a direct welcome/task headline, one sentence of context, and primary actions such as Start a chat, Add sources, and Open knowledge graph. Below it, show only real-data counts for sources, notes, concepts, and local model health, followed by recent-source cards, recent-activity rows, useful agent/tool actions, and an optional graph preview. Hide metrics when empty; never fabricate counts.
- **Chat empty state:** workspace title, one short instruction, a few useful suggested actions (summarize sources, compare ideas, create notes, explore a topic), recent source cards, and a persistent composer.
- **Chat answer:** readable conversation, visible retrieval/tool progress, actionable citation chips (file + page/section), and an honest no-evidence state. Never fabricate sources.
- **Context panel:** upload dropzone, current source list, processing state, selected citation/memory detail, and local model health.
- **Local status:** show “Local mode · Ollama” and current model when local inference is active. Cloud inference is Off by default. If a user explicitly enables a cloud provider, identify it and disclose that prompt/context may leave the machine.

- **Other routes:** notebook list/editor, searchable files/indexing states, graph canvas plus keyboard-accessible list alternative, inspectable/editable memory, and settings/privacy controls.
- **Translate references into product UI:** portfolio cards can become recent-source cards; service rows can become agent/tool or indexing-job rows; progress meters can show actual ingestion/task progress; technical maps can inspire a labeled graph view; pill CTAs can become compact quick actions. Use a carousel only for a genuinely ordered, small set; use ordinary lists otherwise. Do not add testimonials, partner logos, marketing metrics, or decorative contact CTAs.

Interaction and quality

Use reusable components and feature folders; do not build one giant page component. Make controls keyboard- and touch-operable, provide semantic labels and visible focus, and keep citations and graph relationships readable. Keep animation brief, optional, and subordinate; honor `prefers-reduced-motion` and provide static equivalents. Retain native cursor, browser scrolling, text selection, and scrollbars. No scroll hijacking, forced cursors, auto-playing 3D/WebGL, permanent floating decoration, or loader that blocks the workspace. Reflow side panels into drawers/tabs on small screens without horizontal scrolling.

Implementation boundaries

Follow `AGENTS.md` folder ownership and the PRD. Next.js/React/TypeScript/Tailwind only in `apps/web/`. Do not create a single-file HTML app. Core app must work locally and remain useful without cloud keys or external-service availability. No Docker or container files. First inspect the existing UI and project files, then state the smallest planned change before editing. After implementation, run the relevant checks and summarize changed files and checks.

End of Antigravity UI direction prompt.